

Debugging

Advanced Programming, ECE Department, University of Tehran



What is a Bug?

A **bug** refers to an error or fault in a program that causes it to produce incorrect or unintended results. Bugs can arise from a variety of causes.

Debugging is the process of identifying, analyzing, and resolving these issues to ensure that the program functions as expected.



Famous Software Bugs in History

1

Ariane 5 Rocket Explosion (1996)

An overflow error in the navigation system caused the rocket's destruction 37 seconds after launch.

2

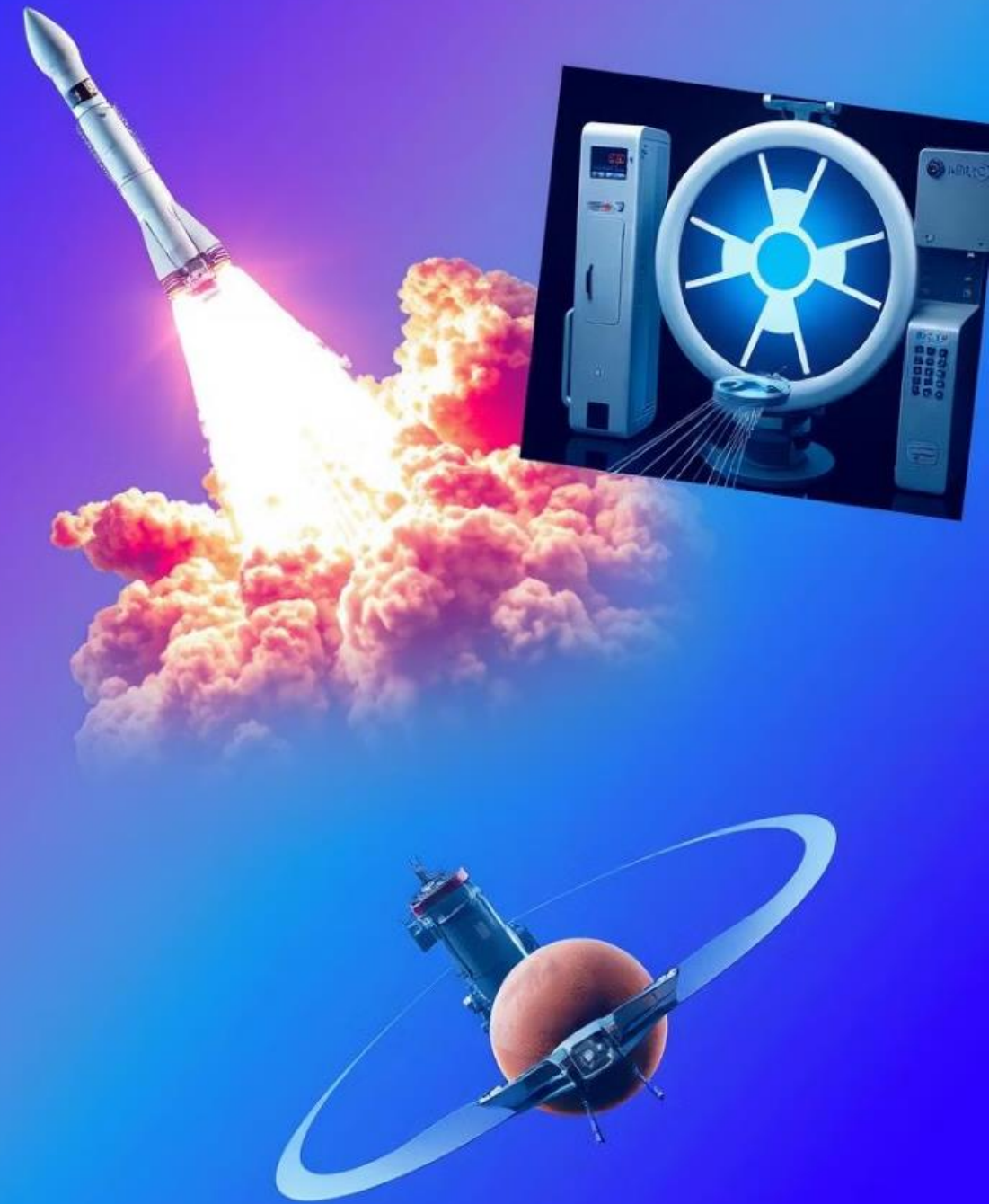
The Y2K Bug: A Millennium Scare

It was a software issue where systems used two digits to represent the year, potentially misinterpreting 2000 as 1900, leading to global efforts to prevent failures.

3

Mars Climate Orbiter (1999)

Mixed units in code caused the spacecraft to disintegrate in Mars' atmosphere.



Types of Bugs in C++

Syntax Errors

These occur when the code does not follow the correct syntax rules of C++. The compiler catches these errors, and the program won't compile. Missing semicolons, unmatched parentheses, or incorrect use of keywords are syntax errors.

```
int main() {  
    std::cout << "Hello, world!" // Missing semicolon here  
}
```

Types of Bugs in C++

Logical Errors

These occur when the program compiles and runs but produces incorrect results due to flawed logic in the code. Miscalculating a formula or placing an incorrect condition in an `if` statement are logical errors.

```
int a = 5, b = 10;

// Logical error: Using '=' instead of '==' in the condition
if (a = b) {
    std::cout << "a is equal to b" << std::endl;
} else {
    std::cout << "a is not equal to b" << std::endl;
}
```

Types of Bugs in C++

Memory Leaks

A memory leak happens when dynamically allocated memory is not properly freed, leading to the program consuming more memory than necessary. Failing to delete dynamically allocated objects is a kind of memory leak.

```
int* ptr = new int[10];
```

```
// Forgot to delete the dynamically allocated array
```

Types of Bugs in C++

Runtime Errors

These bugs occur when the program encounters a problem during execution, such as trying to divide by zero or accessing invalid memory. Out of bound indexing is a kind of runtime error.

```
int arr[5] = {1, 2, 3, 4, 5};
```

```
// Runtime error: Accessing an out-of-bounds array index
```

```
std::cout << arr[10] << std::endl;
```

The Importance of Debugging

Ensures Accuracy

Debugging ensures the program produces correct results by fixing logic and runtime errors.

Boosts Code Quality

It improves code efficiency and stability by removing faulty logic and potential issues.

Improves User Experience

Debugging prevents crashes and enhances reliability, providing users with a smooth experience.

Finding Bugs

Check Error Messages

These can provide valuable information about where the problem occurred, such as the file name, line number, or function where the issue was detected.

Use Print Statements

Sometimes, inserting **print statements** can be a quick way to trace the execution of the program. Printing the values of variables or confirming the program reaches certain sections of code helps in identifying logical or runtime issues.



Finding Bugs

Analyze Code Logic

Review the code for potential logical mistakes, such as incorrect conditions. Sometimes, bugs are caused by code behaving as written, but not as intended.

Implement Unit Testing

Write or run existing unit tests to validate the correctness of individual functions. Unit tests can reveal hidden bugs.



Common Debugging Techniques



Print Debugging

Insert print statements to track execution flow and variable values.

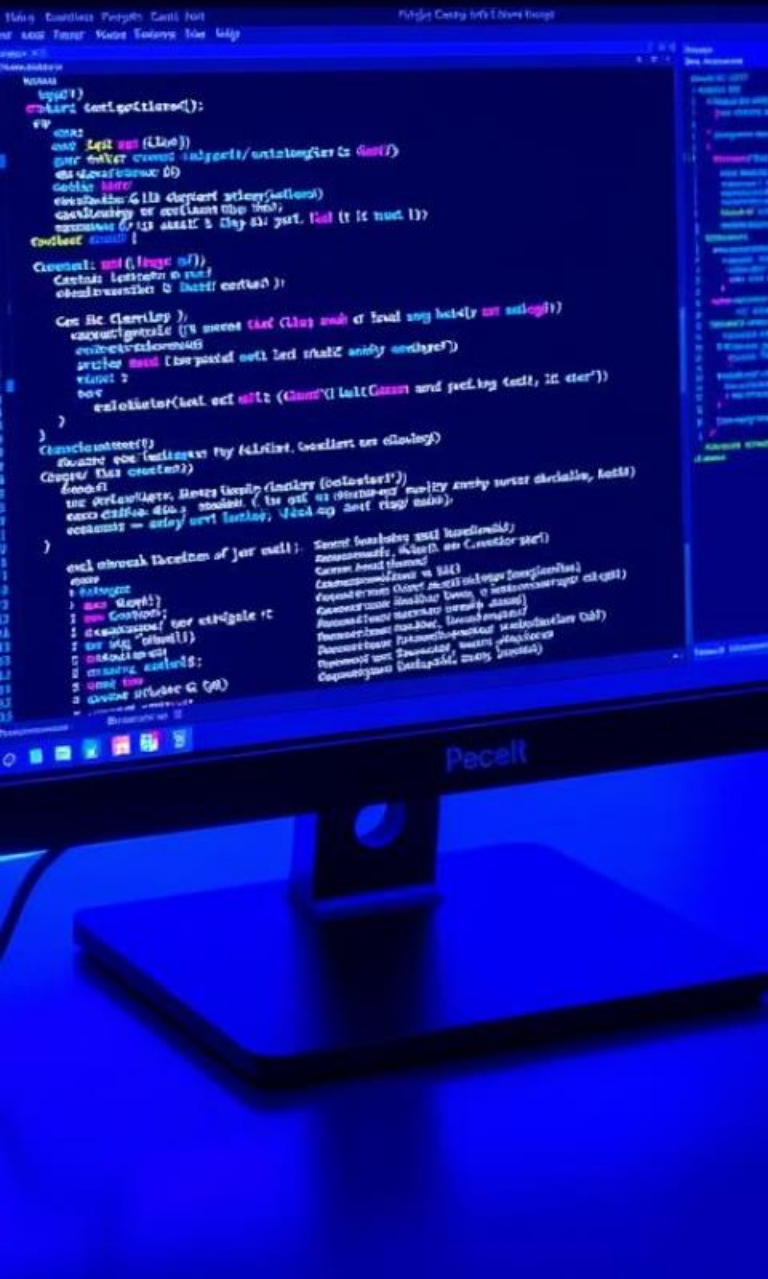
```
std::cout << "Variable x: " << x << std::endl;
```

Common Debugging Techniques



Interactive Debugging

Use tools like GDB to set breakpoints and step through code. You can pause program execution at specific points (breakpoints) and inspect variable states, memory, and the execution flow.



What is GDB?

GDB (GNU Debugger) is a powerful debugging tool for programs written in C, C++, and other languages. It allows developers to run their programs in a controlled environment, where they can inspect variables, set breakpoints, and step through code line by line to diagnose and fix issues.



Getting Started with GDB

To enter GDB, you can simply write "gdb" in the terminal. To exit, type "quit".

```
minerva@minerva-VirtualBox:~$ gdb
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) quit
minerva@minerva-VirtualBox:~$
```

GDB Features

Command	Description
run	Begin program execution under GDB control
kill	Terminate the currently running program being debugged.
break <where>	Pause execution at specific lines
delete <breakpoint #>	Delete specific breakpoint

GDB Features

Command	Description
clear	Remove all breakpoints
enable <breakpoint #>	Enable specific breakpoints that have been previously disabled
disable <breakpoint #>	Deactivate specific breakpoints without deleting them
step	Executes the next line of code, stepping into any function calls encountered

GDB Features

Command	Description
next	Executes the next line of code, but it does not step into functions
finish	Continues execution until the current function returns
print	Evaluates and displays the value of a register or variable
display	Automatically evaluates and displays the value of a register or variable every time the program stops

Debugging Logical Error Using GDB

This program will print whether the average of two numbers is 8 or not.

```
GDB-logical-error.cpp X
GDB-logical-error.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int num1, num2;
7      cout << "Enter first number:" << endl;
8      cin >> num1;
9      cout << "Enter second number:" << endl;
10     cin >> num2;
11     float average = (num1 + num2) / 2;
12     if (average = 8)
13     {
14         cout << "Average is 8!" << endl;
15     }
16     else
17     {
18         cout << "Average is not 8!" << endl;
19     }
20
21     return 0;
22 }
```

Debugging Logical Error Using GDB

Then we test the program with different values.

```
minerva@minerva-VirtualBox:~/Backend$ g++ GDB-logical-error.cpp -o logical_err
minerva@minerva-VirtualBox:~/Backend$ ./logical_err
Enter first number:
9
Enter second number:
7
Average is 8!
minerva@minerva-VirtualBox:~/Backend$ ./logical_err
Enter first number:
8
Enter second number:
8
Average is 8!
minerva@minerva-VirtualBox:~/Backend$ ./logical_err
Enter first number:
7
Enter second number:
5
Average is 8!
minerva@minerva-VirtualBox:~/Backend$ ./logical_err
Enter first number:
20
Enter second number:
50
Average is 8!
minerva@minerva-VirtualBox:~/Backend$
```

Debugging Logical Error Using GDB

We should debug this code. At first compile the code with "ggdb" flag. Then run the "gdb" command.

```
minerva@minerva-VirtualBox:~/Backend$ g++ -ggdb -o logical GDB-logical-error.cpp
minerva@minerva-VirtualBox:~/Backend$ gdb ./logical
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./logical...
(gdb) █
```

Debugging Logical Error Using GDB

Run the program with "run" command.

```
(gdb) run
Starting program: /home/minerva/Backend/logical
Enter first number:
10
Enter second number:
20
Average is 8!
[Inferior 1 (process 35812) exited normally]
(gdb)
```

Debugging Logical Error Using GDB

Set a breakpoint at the beginning of the `main` function to start debugging from the beginning.

```
(gdb) break main
Breakpoint 1 at 0x555555551e9: file GDB-logical-error.cpp, line 5.
(gdb) run
Starting program: /home/minerva/Backend/logical

Breakpoint 1, main () at GDB-logical-error.cpp:5
5      {
(gdb) █
```

Debugging Logical Error Using GDB

```
Breakpoint 1, main () at GDB-logical-error.cpp:5
5      {
(gdb) next
7          cout << "Enter first number:" << endl;
(gdb) n
Enter first number:
8          cin >> num1;
(gdb) n
10
9          cout << "Enter second number:" << endl;
(gdb) n
Enter second number:
10          cin >> num2;
(gdb) n
10
11          float average = (num1 + num2) / 2;
(gdb) info locals
num1 = 10
num2 = 10
average = 4.59163468e-41
(gdb) n
12          if (average == 8)
(gdb) info locals
num1 = 10
num2 = 10
average = 10
(gdb) n
14          cout << "Average is 8!" << endl;
(gdb) info locals
num1 = 10
num2 = 10
average = 8
(gdb) █
```

Debugging Logical Error Using GDB

Then we understand that the problem is in the if statement. We can test it with other values, but this time we will set a breakpoint on the line we suspect is the problem.

```
(gdb) info break
Num      Type      Disp Enb Address              What
1        breakpoint keep y   0x00005555555551e9 in main() at GDB-logical-error.cpp:5
          breakpoint already hit 1 time
(gdb) delete 1
(gdb) info break
No breakpoints or watchpoints.
```

Debugging Logical Error Using GDB

```
(gdb) break GDB-logical-error.cpp:12
Breakpoint 2 at 0x55555555294: file GDB-logical-error.cpp, line 12.
(gdb) run
Starting program: /home/minerva/Backend/logical
Enter first number:
10
Enter second number:
20

Breakpoint 2, main () at GDB-logical-error.cpp:12
12         if (average = 8)
(gdb) info locals
num1 = 10
num2 = 20
average = 15
(gdb) n
14         cout << "Average is 8!" << endl;
(gdb) info locals
num1 = 10
num2 = 20
average = 8
```

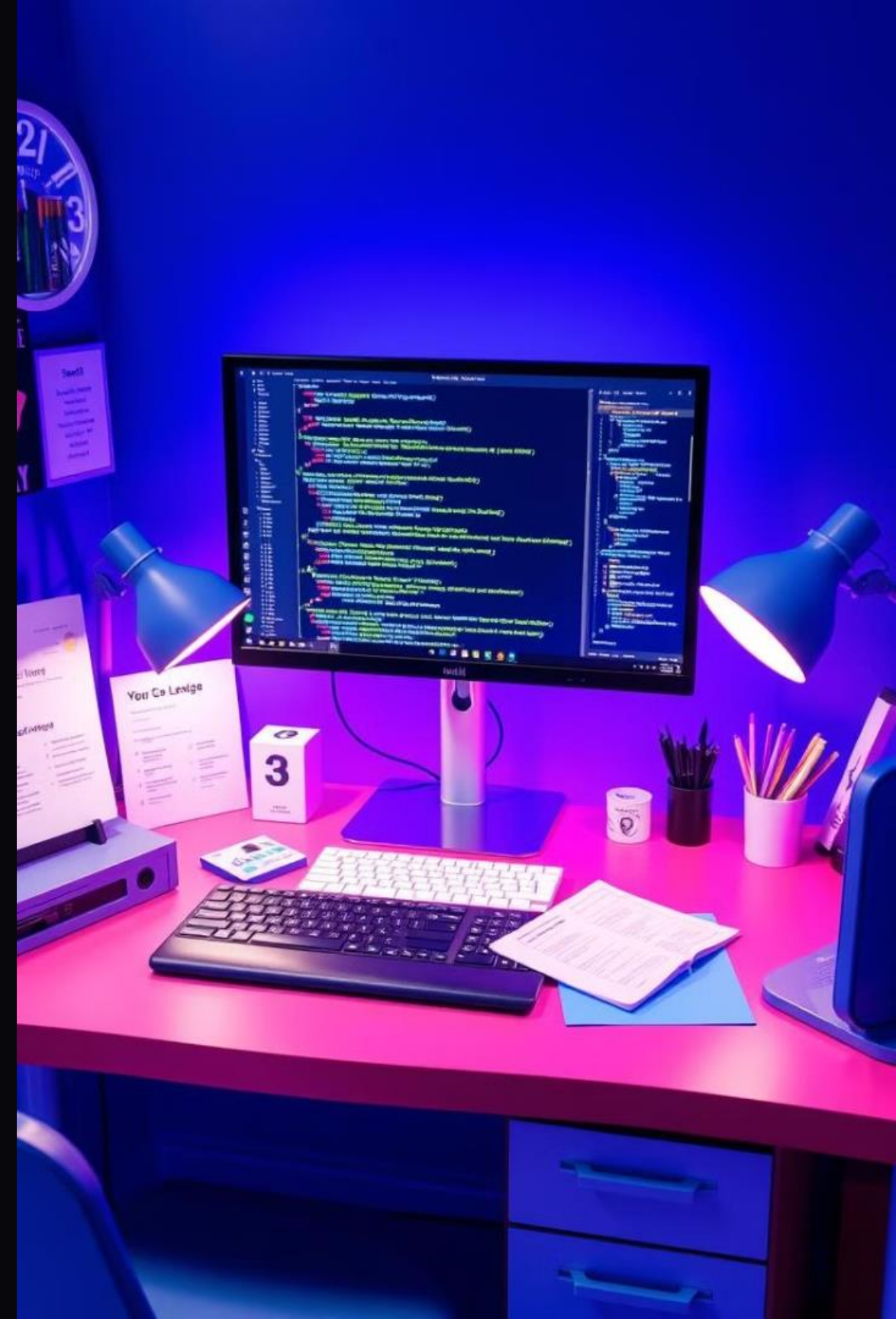
Strategies to Avoid Bugs

Write Clear Code

Use meaningful names and consistent formatting for better readability.

Follow Standards

Adhere to coding standards to maintain consistency and reduce errors.



Strategies to Avoid Bugs

Use Constants

Replace magic numbers with named constants for improved maintainability.

Implement Testing

Regularly test code to catch bugs early in the development process.

